

Parwaaz

AI-Based Multimodal Performing Arts Coach

Facial Expression · Speech Analysis · Singing Pitch & Vocal Technique

Pre-Submission Report | Deep Learning | Group 10

Pratima (230807) · Sejal (230824) · Vanshika (230829) · Khushi (230767) · Shivani (230951)

1. Problem Statement

Aspiring performers; actors, speakers, and singers lack access to affordable, structured training. Hobbyists frequently feel judged for imperfection and abandon their craft early. Existing platforms such as Smule and StarMaker address singing in isolation, while separate tools handle speech or expression independently. No integrated, non-judgmental platform evaluates all three performance dimensions simultaneously. Parwaaz addresses this gap: a multimodal AI coaching system that analyses visual and audio cues together and delivers structured, interpretable post-performance feedback without a human trainer.

2. Objective

Build an MVP that evaluates user performance across three AI-powered dimensions:

- Module 1 — Facial Expression Analysis (acting feedback via temporal Siamese network on CREMA-D)
- Module 2 — Speech Accuracy, Pronunciation & Fluency (Whisper fine-tuned on Hindi + Hindi Sentence Transformer + Prosody Analysis)
- Module 3 — Singing Pitch, Rhythm & Vocal Technique (CREPE + DTW pipeline + trained classifier)

All modules integrate into a FastAPI pipeline returning one unified evaluation report per session, designed for post-performance feedback on commodity hardware.

3. System Architecture

A FastAPI backend receives user media, routes it across three parallel modules, and merges structured JSON outputs. The frontend (React/Next.js) handles upload, recording, and visualisation.

Component	Input / Output	Responsibility
User Upload	Video (.mp4/.webm) + MP3	Frontend — MediaRecorder + file picker
FastAPI Backend	Raw media → stems + frames	ffmpeg splits; routes to all modules
Module 1	Frames → emotion JSON	Training temporal Siamese network and intensity validation
Module 2	Audio → transcript + scores	Whisper + Sentence Transformer
Module 3	Audio → pitch + technique JSON	CREPE + DTW + VocalSet Classifier
Aggregation	3 × JSON → unified report	/analyze merges all outputs
Frontend	Report dict → visual dashboard	Pitch chart, emotion timeline, WER view

4. Module 1 — Facial Expression Analysis

Deep Learning Architecture: Siamese Temporal Network Instead of simple frame-based classification, this module now utilizes a reference-based evaluation pipeline to assess the quality of acting against a target performance.

- Feature Extraction:** A **CNN Encoder** (pretrained backbone) processes sampled frames from both the User Video and the Reference Video.

- **Temporal Modeling:** A **Temporal Transformer** with shared weights captures the emotional progression and timing of expressions over the duration of the clip.
- **Evaluation Mechanism:** A **Cross-Attention Comparator** (CoRe-inspired) aligns and compares the user's temporal features against the reference.
- **Scoring:** A **Score Distribution Head** (MUSDL-inspired) outputs a final acting score (0–10) and generates granular, per-segment feedback.

Training & Dataset Methodology

- **Primary Dataset: CREMA-D** (Crowd-sourced Emotional Multimodal Actors Dataset).
- **Composition:** Utilized for its high-quality emotional acting clips across various intensities, providing the ground truth for "ideal" emotional expression.
- **Validation:** Cross-validated using **CK+** sequences to ensure the encoder generalizes well to diverse facial morphologies

Output Field	Description
acting_score	Total score (0–10) based on alignment with reference intensity and timing
segmental_feedback	Qualitative notes on specific timestamps (e.g., "Increase intensity at 0:05")
temporal_alignment	Data for the frontend to show user vs. reference emotion curves

5. Module 2 — Speech Accuracy & Pronunciation

Deep Learning Components

Whisper (OpenAI) — Encoder-Decoder Transformer. Trained on 680,000 hours of multilingual audio. Returns word-level timestamps and decoder softmax confidence per token.

Sentence Transformers — all-MiniLM-L6-v2. Distilled BERT model. Encodes each word into a 384-dimensional embedding. Cosine similarity between reference and user embeddings gives a per-word pronunciation score (0–1).

$$\text{word_score} = 0.7 \times \text{cosine_similarity} + 0.3 \times \text{whisper_confidence}$$

WER (jiwer): Levenshtein alignment classifies each word as substitution, deletion, or insertion. Classical metric but standard NLP practice paired with the DL components above.

Metric	Method
Word Error Rate (WER)	Levenshtein alignment; substitutions + deletions + insertions / total words
Per-word pronunciation	Cosine similarity of 384-dim word embeddings (0–1 scale)
Overall speech score	Weighted fusion: $0.7 \times \text{cosine sim} + 0.3 \times \text{Whisper decoder confidence}$
Pause Duration	Inter-word gap from Whisper timestamps; gaps >500ms flagged as hesitation
Speech Rate (WPM)	Word count / segment duration from Whisper timestamps; target range 100–160 WPM
Filler Word Count	Token-match over Whisper transcript against filler vocabulary (“um”, “uh”, “matlab”, “toh” etc.); counted as insertions in WER + standalone count

Hindi Fine-Tuning Strategy

The default Whisper (base) and all-MiniLM-L6-v2 models are trained primarily on English-dominant corpora. For Parwaaz’s target use case - evaluating Hindi speech by Indian performers both components require adaptation.

Whisper Fine-Tuning:

Whisper-small (244M) is used with the encoder frozen to retain pretrained Hindi acoustic features and avoid overfitting on around 94903 clips. Only the decoder is fine-tuned using cross-entropy loss with a low learning rate (1e-5) and warmup + linear decay. Training runs for ~3 epochs (~2–3 hours on T4 GPU).

Sentence Transformer Update:

all-MiniLM-L6-v2 is replaced with l3cube-pune/hindi-sentence-similarity-sbert (same 384-dim output). Cosine similarity remains unchanged, but embeddings improve due to Hindi-specific training, better capturing phonetic distinctions.

6. Module 3 — Singing Pitch, Rhythm & Vocal Technique

Reference-Based Evaluation Pipeline (Pretrained DL Models)

The user uploads any MP3 reference (≤60s). The system builds an acoustic profile from it, then compares the user’s live recording against that profile using three pretrained deep learning models as feature extractors.

Demucs htdemucs — Hybrid Transformer-Conv Encoder-Decoder. Trained on MUSDB18. Separates vocals from instrumentation. Without this step, instrumental harmonics corrupt pitch estimation.

CREPE (tiny) — 6-Layer CNN. Maps raw audio frames to a 360-bin pitch probability distribution, outputting frequency (Hz), time, and confidence per frame. Frames with confidence < 0.5 are flagged as unvoiced and excluded from all metric calculations.

Whisper base — Transformer STT - Transcribes reference and user audio with word-level timestamps for WER-based lyrics scoring.

Alignment: Dynamic Time Warping (DTW) aligns the user’s pitch timeline to the reference non-linearly, handling tempo drift. Global key correction (median pitch offset → semitone shift) compensates for users singing in a different key before frame-by-frame comparison.

Metric	Method / Interpretation
Pitch Score (0–100)	% voiced frames within ±50 cents after DTW + key correction
Mean Pitch Error	Mean absolute cent deviation on voiced frames
Pitch Stability	Std dev within sustained segments — low = controlled delivery
Rhythm Score (0–100)	Mean onset deviation (ms); tempo ratio flags rushing or dragging
Lyrics Score	WER between Whisper transcriptions of reference and user
Key Offset	Semitones user sang above / below reference key

Final = 0.50 × Pitch + 0.30 × Rhythm + 0.20 × Lyrics

[renormalised to 0.625/0.375 if lyrics disabled]

7. Dataset Strategy

Module	Dataset	Purpose
Module 1	CREMA-D (7,442 clips) + CK+	CREMA-D (7,442 clips) + CK+
Module 2	Whisper-small fine-tuned on AI4Bharat Kathbath Hindi (~94903 clips); Hindi SBERT (l3cube-pune)	Hindi ASR fine-tuning (transfer learning, decoder-only); Hindi embedding calibration; WER + fluency threshold validation

Module 3	VocalSet excerpts + 15–20 custom recordings (good/bad/fast)	Pipeline testing & metric validation
-----------------	---	--------------------------------------

8. Technology Stack

Category	Tools / Libraries
Facial Analysis	PyTorch, OpenCV, CREMA-D , CK+, timm (for pretrained backbones)
Speech Analysis	openai-whisper (base), sentence-transformers (all-MiniLM-L6-v2), jiwer
Singing — Evaluation	crepe (tiny), demucs (htdemucs), librosa ≥ 0.10 , soundfile
Alignment / Numerics	numpy, scipy, librosa.sequence.dtw
Backend / Infra	FastAPI, ffmpeg, Python 3.10+
Frontend	React / Next.js, Chart.js or Recharts
Compute	Kaggle free T4 GPU (30 hrs/week) / Google Colab/ local / University's GPU (whenever available)

9. Integration & Final Output

Each module exposes a single Python callable with a defined JSON output contract. The FastAPI aggregation layer merges all three into one unified report per session:

- Annotated video with per-frame emotion overlays and emotion timeline graph
- Speech report: overall score, per-word pronunciation, WER substitution/deletion/insertion breakdown
- Singing report: pitch / rhythm / lyrics / stability scores, key offset, pitch contour chart,, and natural-language feedback strings

Integration contracts pre-defined: Module 1 → predict(frames_dir) → dict; Module 3 → analyze_pitch(ref_path, user_path) → dict.

10. Current Progress & Next Steps

Module	Completed	In-Progress
Module 1	Architecture designed; CREMA D downloaded; CNN backbone selected(Resnet18)	Train the EmotionCNN Encoder and Temporal Transformer on CREMA-D sequences
Module 2	Whisper + Sentence Transformer pipeline defined; scoring formula finalised	IFine-tune Whisper-small decoder on Kathbath Hindi; swap SBERT to Hindi model; implement fluency/prosody pipeline; build eval set; calibrate thresholds
Module 3	Methodology documented; Architecture designed; Pre-processing functions made; CREPE + DTW + Filtering	Custom Dataset creation; Vocal separation (Demucs); rhythm analysis; lyrics scoring
Integration	Output contracts defined for all three modules	Build FastAPI aggregation layer; wire frontend report page